

Mudcard

- **The transformer is to get the categorical values into some number values so a machine learning model could be run on them right?**
 - It's a bit more general than that.
 - Yes, transformers are used to convert categorical values into numerical values (like categorical features into one-hot encoded dummy variables and ordinal features into numbers) but transformers are also used to bring all numerical features into the same order of magnitude.
- **I am still unsure about why is encoder necessary.**
 - Call a sklearn machine learning model like logistic regression on the raw census data.
 - You will get an error message because the feature matrix contains non-numerical (string) values
 - We use transformers to transform the feature matrix such that it contains numbers only

Lecture 7: Linear models

By the end of this lecture, you will be able to

- describe the main components of any ML algorithm
- describe how linear regression works
- describe how logistic regression works

The supervised ML pipeline

0. Data collection/manipulation: you might have multiple data sources and/or you might have more data than you need

- you need to be able to read in datasets from various sources (like csv, excel, SQL, parquet, etc)
- you need to be able to filter the columns/rows you need for your ML model
- you need to be able to combine the datasets into one dataframe

1. Exploratory Data Analysis (EDA): you need to understand your data and verify that it doesn't contain errors

- do as much EDA as you can!

2. Split the data into different sets: most often the sets are train, validation, and test (or holdout)

- practitioners often make errors in this step!
- you can split the data randomly, based on groups, based on time, or any other non-standard way if necessary to answer your ML question

3. Preprocess the data: ML models only work if X and Y are numbers! Some ML models additionally require each feature to have 0 mean and 1 standard deviation (standardized features)

- often the original features you get contain strings (for example a gender feature would contain 'male', 'female', 'non-binary', 'unknown') which needs to be transformed into numbers
- often the features are not standardized (e.g., age is between 0 and 100) but it needs to be standardized

4. Choose an evaluation metric: depends on the priorities of the stakeholders

- often requires quite a bit of thinking and ethical considerations

5. Choose one or more ML techniques: it is highly recommended that you try multiple models

- start with simple models like linear or logistic regression
- try also more complex models like nearest neighbors, support vector machines, random forest, etc.

6. Tune the hyperparameters of your ML models (aka cross-validation or hyperparameter tuning)

- ML techniques have hyperparameters that you need to optimize to achieve best performance

- for each ML model, decide which parameters to tune and what values to try
- loop through each parameter combination
 - train one model for each parameter combination
 - evaluate how well the model performs on the validation set
- take the parameter combo that gives the best validation score
- evaluate that model on the test set to report how well the model is expected to perform on previously unseen data

7. Interpret your model: black boxes are often not useful

- check if your model uses features that make sense (excellent tool for debugging)
- often model predictions are not enough, you need to be able to explain how the model arrived to a particular prediction (e.g., in health care)

A few words on evaluation metrics

- we use **accuracy** as our classification metric for now
- we use **RMSE** or **MSE** as our regression metric for now
- we will have two lectures dedicated to evaluation metrics during the second half of the term

Supervised ML algorithms, part 1

By the end of this lecture, you will be able to

- **describe the main components of any ML algorithm**
- describe how linear regression works
- describe how logistic regression works

Supervised ML algorithms: three parts

- 1. a **mathematical model** (f) is used to convert the feature values into a prediction

$f(X_i) = y_i$, where X_i is the i th data point in our sample. X_i is a vector and y_i is a number. - f is your supervised ML algorithm - it usually has a number of intrinsic parameters

- 2. the optimization algorithm minimizes a metric called **the loss function**
 - the loss function is used to determine the best intrinsic parameters of one model based on the training data
 - it is not the same as the evaluation metric
 - you use the evaluation metric to compare various models
 - the model uses the loss function to find the best values of its intrinsic parameters
 - keep in mind though that the same metric can be used as the loss function and the evaluation metric (e.g., MSE in regression) but that's not necessarily the case (e.g., the loss function is MSE but you use R^2 as an evaluation metric).
- 3. an **optimization algorithm** is used to determine the intrinsic parameter values given the training set and the loss function
 - there are various algorithms
 - e.g., analytical solutions, brute force, gradient descent, backpropagation

Supervised ML algorithms, part 1

By the end of this lecture, you will be able to

- describe the main components of any ML algorithm
- **describe how linear regression works**
- describe how logistic regression works

Linear Regression

```
In [ ]: from sklearn.linear_model import LinearRegression # import the model
LinReg = LinearRegression() # initialize a simple linear regression model
LinReg.fit(X_train,y_train) # we will learn now what happens when you issue this line in regression
```

- This is the **mathematical model**:

$$f(X_i) = y_i' = w_0 + w_1 X_{i1} + w_2 X_{i2} + \dots = w_0 + \sum_{j=1}^d w_j X_{ij}$$

$$f(X_i) = y_i' = \langle w, X_i \rangle$$

where y_i' is the prediction of the linear regression model and w are parameters or weights of the linear model.

- The **loss function** is the mean squared error:

$$MSE(y, y') = \frac{1}{m} \sum_{i=1}^m (y_i - y_i')^2 = \frac{1}{m} \sum_{i=1}^m (y_i - \langle w, X_i \rangle)^2$$

- We will explore two **optimizers**:
 - brute force approach
 - least squares method

Let's generate some data

```
In [ ]: # load packages and generate data
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import matplotlib
from sklearn.datasets import make_regression
matplotlib.rcParams.update({'font.size': 11})

# fix the seed so code is reproducible
np.random.seed(1)

# generate n_samples points
n_samples = 30

# generate data
X, y, coef = make_regression(n_samples = n_samples, n_features = 1, coef = True, noise= 10, bias=0)
print(coef) # the coefficients of the underlying linear model, the bias is 0.
print(np.dot(X,coef)[:0]) # noise is added to the label
print(y)
df = pd.DataFrame()
df['x1'] = X[:,0]
df['y'] = y
print(df.head())
df.to_csv('../data/regression_example.csv', index=False)
```

```
In [ ]: def model_linreg(X,w):
    if len(np.shape(w)) != 2:
        w = np.array(w)[np.newaxis,:] # a numpy trick to make the dot product work

    y_pred = w[0,0] + w[0,1:].dot(X.T) # intercept + <w,x>

    return y_pred

def loss_function_MSE(X,y_true,w):
    """
    Take in a numpy array X,y_true, w and generate the loss function
    of using w as parameter in a linear regression model
    """
    m = len(y)
    w = np.array(w)[np.newaxis,:] # just a numpy trick to make the dot product work in predict
    y_pred = model_linreg(X,w)
    loss = (1/m) * np.sum(np.square(y_true-y_pred)) # this is MSE
    return loss
```

$$y_i = w_0 + x_{i1} w_1$$

- w_0 is the intercept
- w_1 is the slope

We are looking for the best fit line!

For a given w vector, the loss function returns the MSE.

```
In [ ]: w = [0, 0] # intercept is w[0], the slope is w[1]

plt.figure(figsize=(5,3))
plt.scatter(X,y,label='true values')
plt.plot(X,model_linreg(X,w),label='predictions',color='r')
plt.title('the loss function: '+str(np.around(loss_function_MSE(X,y,w),2)))
plt.xlabel('x1')
plt.ylabel('y')
plt.legend()
plt.savefig('../figures/line_fit.png',dpi=300)
plt.show()
```

What we want:

- Find the w vector that minimizes the loss function! That's our best fit model.

$$\|w\| = \text{argmin}_{\|w\|} \sum_{i=1}^m (y_i - \langle w, X_i \rangle)^2$$

How we do it:

- Brute force approach
 - create a grid of $w[0]$ and $w[1]$ values
 - loop through all w vectors on the grid
 - find the w vector that comes with the smallest loss

```
In [ ]: n_vals = 101

w0 = np.linspace(-100,100,n_vals) # the intercept values to explore
w1 = np.linspace(-100,100,n_vals) # the slope values to explore

loss = np.zeros([len(w0),len(w1)]) # the loss function's value for each w

# loop through all intercept-slope combinations and calculate the loss function
for i in range(n_vals):
    for j in range(n_vals):
        w = [w0[i],w1[j]]
        loss[i,j] = loss_function_MSE(X,y,w)

print('min(loss):',np.min(loss))
min_coors = np.unravel_index(loss.argmin(),np.shape(loss))
print('best intercept:',w0[min_coors[0]])
print('best slope:',w1[min_coors[1]])
```

```
In [ ]: plt.figure(figsize=(6,5))
ax = plt.gca()
extent = (np.min(w0),np.max(w0),np.min(w1),np.max(w1))
fig = ax.imshow(loss.T,origin='lower',extent=extent,vmin=0)
plt.colorbar(fig,label='loss function')
ax.contour(w0,w1,loss.T,levels=5,colors='black')
plt.scatter(w0[min_coors[0]],w1[min_coors[1]],c='r')
ax.xaxis.set_ticks_position("bottom")
plt.xlabel('intercept')
plt.ylabel('slope')
plt.title('loss function parameter space')
plt.tight_layout()
plt.savefig('../figures/loss_function.png',dpi=300)
plt.show()
```

A much better optimizer: least squares method

- brute force is not used in practice, too computationally expensive
- linear regression can be solved analytically meaning that we can get an exact solution
- this is called the least squares method

```
In [ ]: def least_squares(X,y):
    # append a column of 1s for the bias term w_0
    X_bias = np.concatenate((np.ones((X.shape[0], 1)),X), axis=1)
    # matrix A is <X.T,X>
    A = (X_bias.T).dot(X_bias)
    # vector b is <X.T,y>
    b = (X_bias.T).dot(y)
    # the optimal w is A^(-1)b
    # this is an exact solution
    w = np.linalg.inv(A).dot(b)
    return w

w = least_squares(X,y)
print(w)
```

```
In [ ]: # visualize the solution
plt.figure(figsize=(5,4))
plt.scatter(X,y,label='true values')
plt.plot(X,model_linreg(X,w),label='predictions',color='r')
plt.title('the loss function: '+str(np.around(loss_function_MSE(X,y,w),2)))
plt.xlabel('x1')
plt.ylabel('y')
plt.legend()
plt.savefig('../figures/line_fit.png',dpi=300)
plt.show()
```

Quiz 1

Supervised ML algorithms, part 1

By the end of this lecture, you will be able to

- describe the main components of any ML algorithm
- describe how linear regression works
- **describe how logistic regression works**

Logistic regression

```
In [ ]: from sklearn.linear_model import LogisticRegression
LogReg = LogisticRegression() # initialize a simple logistic regression model
LogReg.fit(X_train,y_train) # we will learn what happens when you issue this line in classification
```

- name is misleading, logistic regression is for classification problems!
- the model:

$$y_i = \frac{1}{1+e^{-z}}$$

where

$$z = w_0 + \sum_{j=1}^d w_j x_{ij} = \langle w, X_i \rangle$$

$f(z) = \frac{1}{1+e^{-z}}$ is the **sigmoid function**.

- a sigmoid function is applied to the output of a linear regression model

```
In [ ]: def sigmoid(z):
        return 1/(1+np.exp(-z))

z = np.linspace(-7,7,50)

plt.figure(figsize=(5,3))
plt.plot(z,sigmoid(z))
plt.xlabel('input to sigmoid function (z)')
plt.ylabel('predicted probability (f(z))')
plt.title('sigmoid transformation')
plt.savefig('../figures/sigmoid_trans.png',dpi=300)
plt.show()
```

The loss function

- the logloss metric is used as a loss function in logistic regression

$$\mathcal{L}(w) = - \frac{1}{m} \sum_{i=1}^m [y_i \ln(y_i) + (1-y_i) \ln(1-y_i)]$$

$$\mathcal{L}(w) = - \frac{1}{m} \sum_{i=1}^m [y_i \ln(\frac{1}{1+e^{-w_0 + \sum_{j=1}^m w_j x_{ij}}}) + (1-y_i) \ln(1-\frac{1}{1+e^{-w_0 + \sum_{j=1}^m w_j x_{ij}}})]$$

- y_i is the predicted probability of the **positive class**
- two scenarios:
 - $y_{\text{true}} = 0$ - left term disappears
 - $y_{\text{true}} = 1$ - right term disappears

Let's generate some data!

```
In [ ]: from sklearn.datasets import make_classification
X, y = make_classification(n_samples=100, n_features=2, n_informative=2, n_redundant=0,
                          n_repeated=0, n_clusters_per_class=1, class_sep=0.5, random_state=37)

df = pd.DataFrame()
df['x1'] = X[:,0]
df['x2'] = X[:,1]
df['y'] = y
print(df.head())
df.to_csv('../data/classification_example.csv', index=False)
```

```
In [ ]: plt.figure(figsize=(5,3))
plt.axis('equal')
plt.scatter(X[:,0],X[:,1],c=y)
plt.xlabel('feature 1')
plt.ylabel('feature 2')
plt.savefig('../figures/classification_toy.png',dpi=300)
plt.show()
```

```
In [ ]: def sigmoid(z):
        return 1/(1+np.exp(-z))

def model_logreg(X,w):
    w = np.array(w)[np.newaxis,:]
    y_pred = sigmoid(w.dot(X.T)).ravel()
    return y_pred

def loss_function_logloss(X,y_true,w):
    """
    Take in a numpy array X,y_true, w and generate the loss function
    of using w as parameter in a linear regression model
    """
    m = len(y)
    y_pred = model_logreg(X,w)
    loss = -(1/m) * np.sum(y_true*np.log(y_pred)+ (1-y_true)*np.log(1-y_pred))
    return loss
```

```
In [ ]: w = [1.8,-1.5] # notice that w has two components and we have two features. what does this mean?

n_vals = 31
x0 = np.linspace(-3,3,n_vals) # the intercept values to explore
x1 = np.linspace(-3,3,n_vals) # the slope values to explore

pred_prob = np.zeros([len(x0),len(x1)]) # the loss function's value for each w

# loop through all intercept-slope combinations and calculate the loss function
for i in range(n_vals):
    for j in range(n_vals):
        point = np.array([x0[i],x1[j]]).reshape(1,-1)
        pred_prob[i,j] = model_logreg(point,w)[0]

plt.figure(figsize=(5,4))

# visualize the predicted probabilities
xx, yy = np.meshgrid(x0,x1)
plt.contourf(xx,yy,pred_prob.T,origin='lower',vmin=0.0,vmax=1.0,levels=np.arange(0,1.01,0.01))
plt.colorbar(label='predicted probability')
plt.contour(xx, yy, pred_prob.T,vmin=0,vmax=1,levels=[0.5],colors=['lime'],linewidths=4)

# add the datapoints
plt.scatter(X[:,0],X[:,1],c=y,s=50,cmap='grey')
plt.title('the loss function: ' + str(np.around(loss_function_logloss(X,y,w),4)))
plt.xlabel('feature 1')
plt.ylabel('feature 2')

# add the w vector
plt.arrow(0,0,w[0],w[1],width=0.1,length_includes_head=True,color='r',label='w vector')
plt.legend()

plt.savefig('../figures/classification_toy_model.png',dpi=300)
plt.show()
```

Quiz 2

Optimizer

- no exact solution like least squares because the non-linear sigmoid function complicates things

Gradient descent

- start with arbitrary initial w values and the loss function L
- repeat until convergence:

$$w_j := w_j - \eta \frac{\partial L(w)}{\partial w_j},$$

where $\frac{\partial L(w)}{\partial w_j}$ is the gradient of the loss function at the current w location and η is the learning rate.

- the gradient tells us which way the loss function is the steepest
- the learning rate tells us how big of a step we take in that direction

```
In [ ]: def gradient_descent(X,y_true,w, learning_rate=0.01,iterations=100):
    """
    X    = Matrix of X
    y    = Vector of Y
    w    = Vector of ws
    learning_rate
    iterations = no of iterations

    Returns the final w vector and array of loss history over no of iterations
    """
    m = len(y_true)

    loss_history = np.zeros(iterations)
```

```

w_history = np.zeros([iterations+1,np.shape(w)[0]])
w_history[0] = w

for it in range(iterations):

    # calculate gradients here
    # predict
    y_pred = model_logreg(X, w)
    # error
    error = y_pred - y_true
    # gradient of log-loss w.r.t. w
    delta_w = (1/m) * X.T.dot(error)
    # gradient step
    w = w - learning_rate * delta_w

    # update histories
    w_history[it+1] = w
    loss_history[it] = loss_function_logloss(X,y_true,w)

return w, loss_history, w_history

```

```

In [ ]: w,loss_history,w_hist = gradient_descent(X,y,[-10,-10],0.02,10000)
print(w) # this is w once the iteration is completed

```

```

In [ ]: plt.figure(figsize=(5,4))

plt.plot(w_hist[:,0],w_hist[:,1],color='r',label='path')
plt.scatter(w_hist[:,200,0],w_hist[:,200,1],c='r')
plt.xlabel('w0 component')
plt.ylabel('w1 component')
plt.title('w history and path')
plt.tight_layout()
plt.savefig('../figures/gradient_path.png',dpi=300)
plt.show()

```

```

In [ ]: plt.figure(figsize=(5,3))
plt.plot(loss_history)
plt.semilogy()
plt.ylabel('loss function')
plt.xlabel('nr of iterations')
plt.title('loss function history')
plt.savefig('../figures/loss_hist.png',dpi=300)
plt.show()

```

DO NOT USE MY `gradient_descent` FUNCTION!

- it is for illustration purposes only
- it is much slower than the sklearn implementation!

Mud card